Data Structure Using C++

Revision

Dr. Ali Hamzah

Instructor

❑ Dr. Ali Abdullah Hamzah

- – Allawi.cs@gmail.com
- – ***Office**: Mondays and Wednesday*
- – ***Phone**:*

❑ Research interests:

- – AI,NLP, Information security based on linguistics
- – Data Analytics

Grading

Type	Grades
▪ Quizzes & Assignments& Attend	20
▪ Mid term Exam	20
▪ project	15
▪ Final exam	40
Total	100

Programming languages and program life cycle

- Programming Languages is a process of problem solving
- **Steps:**
  - Analysis the problem
    - Outline the problem and outline its requirements
    - Design steps(algorithms) to solve the problem
  - Implement the algorithm
    - Implement the algorithm in code
    - Verify the algorithm works
  - Maintenance
    - Use and modify the program

Analysis the problem

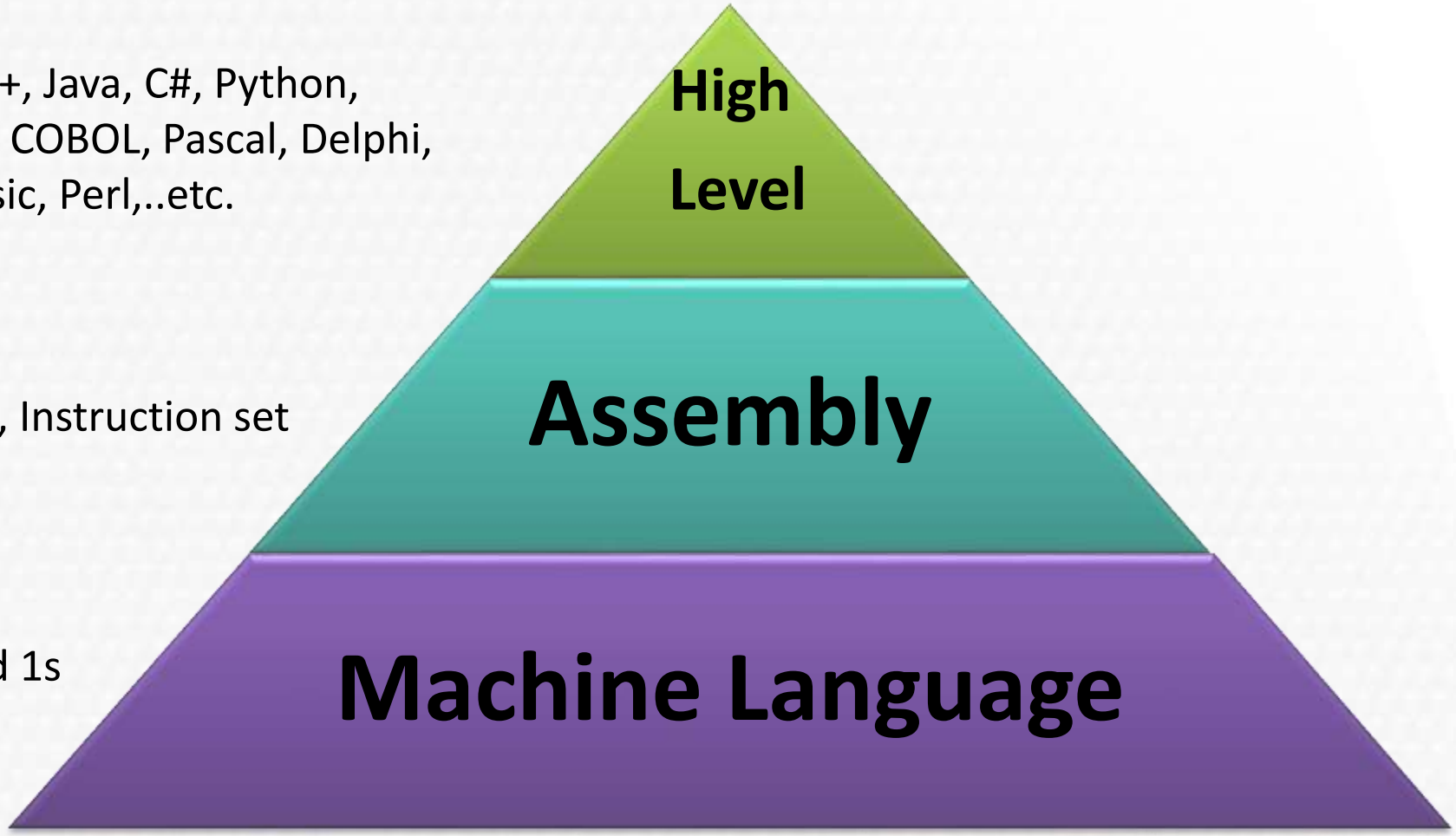
- **Understanding the problem requirements**
  - Does the program require user interaction?
  - Does the program require data processing?
  - Does the user require output?
  - ➢ Ex: grads system
    - ***University ID as input***
    - ***Printing result as output***
- ➢ **The complex problem**
  - Dividing to sub problem and analyze each part (as above)

Programming Languages

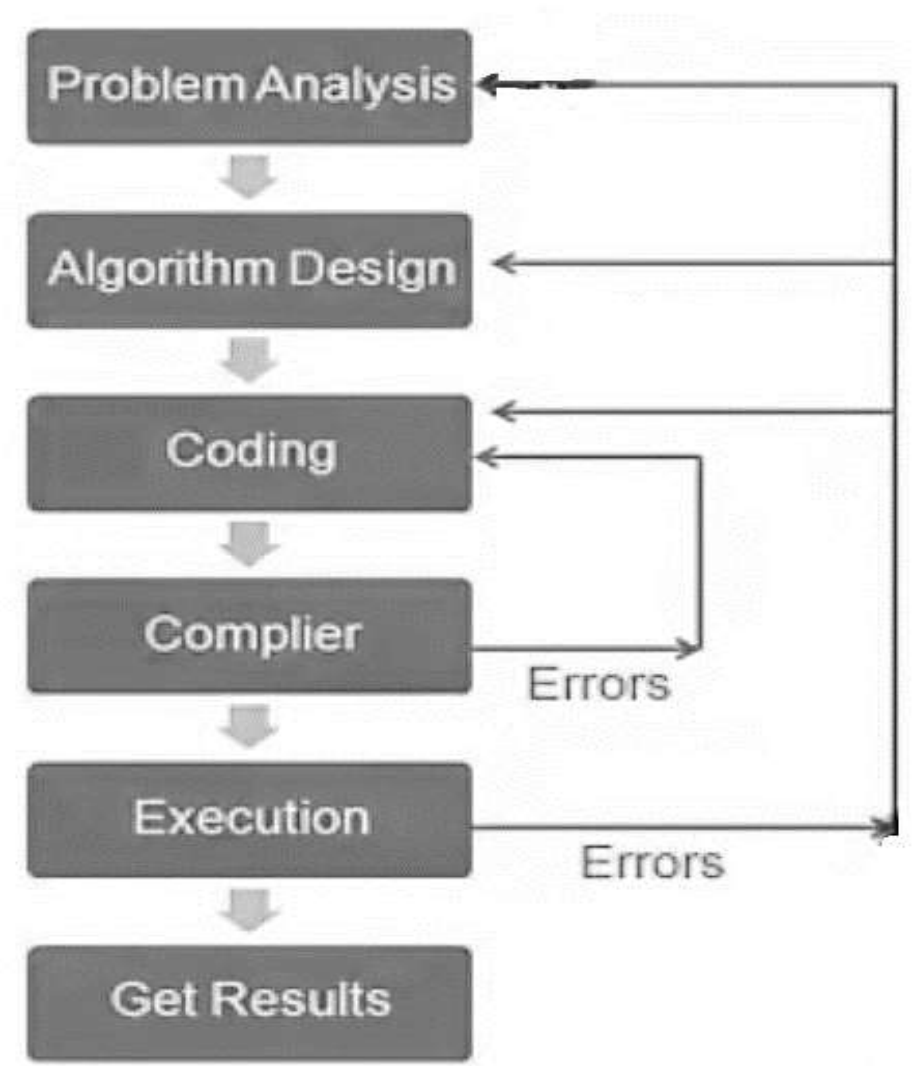
C,C++, Java, C#, Python,
Ada, COBOL, Pascal, Delphi,
VBasic, Perl,..etc.

MIPS, Instruction set

0s and 1s



Problem analysis & code execution cycle



Your first C++ Program ☺

- You now know the enough basics to create and compile a program.
- The program will use the Standard C++ iostream classes.
- In this simple program, a stream object will be used to print a message on the screen.
- To send data to standard output, you use the operator <<
- Example:
  - cout<< "Welcome to FCI";

“Hello, world!”

And now finally, the first program:

```
// helloworld.cpp : Defines the entry point for the console application.
/*
    Written on Ocober, 8th  2023
*/
#include <iostream>
using namespace std;
int main()
{
    cout<<"Hello, world!"
        <<" this is my first day in the "
        <<"programming campaign"<<endl;
return 0;
}
```

Working with Header files

- In order to avoid writing too much code, some pieces of code were wrapped within a *function*.

- A *function* is an atomic unit of code, which performs specific action, that can be placed in different files, we'll talk about it later.

- In C++, there are a lot of standard functions to help you while writing your code, grouped together in an object called "*Library*".

Working with Header files cont'd

- There are a lot of standard libraries, that you can include within your code; in order to be able to use the supported functions.
- Most libraries are declared in a device called ***header file***.
- A programmer who creates a library, provides a header file for it.
- To include and use a header file we use the **#include** directive.

Working with Header files (cont'd)

- The input/output functions are for example in the standard C++ library. To use them, you just include the **<iostream.h>** header file.
- In C++, each set of C++ definitions in a library is "wrapped" in a ***namespace***.
- If you simply include a header file and use functions from the header without include its namespace, you will get strange sounding errors ☹ .

Using namespace std;

- To tell the compiler: "I want to use the functions in this namespace.."
- This keyword is **using**
- All the standard C++ libraries are wrapped in a single namespace which **std** (for "standard")

- #include <iostream.h>

  means

  #include <iostream>
  Using namespace std;

Fundamentals of Program Structure (program body)

- When the programs starts, it executes initialization code and calls a special function, "**main()**". You put the primary code for the program here.
- As will be mentioned later, the functions have parameters and return types.
- In C++, **main()** always has return type of **int**.

Fundamentals of Program Structure

- Note that, Compilers ignores:
  - Whitespaces
  - Newlines, so it has a way to determine the end of statement, with is the semi colon (;)
  - Comments (/* …*/ , //)

Reading Input

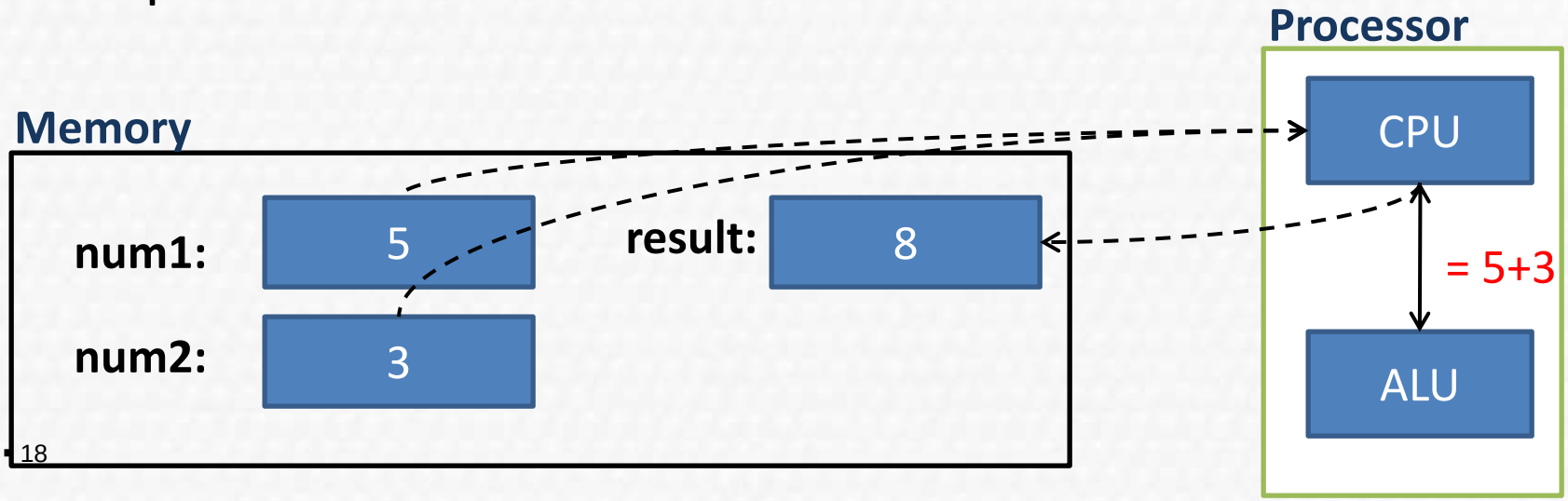
- The iostream's operator used with cin is >>
- This operator waits for the same kind of input as its argument.
- If you give it an integer argument, it waits for an integer from the console.

Reading input

```
// helloworld.cpp : Defines the entry point for the console application.
/*
Written on Januray, 28th  2011
*/
#include <iostream>
using namespace std;
int main()
{
    int num1, num2;
    cout<<"Enter two numbers to get their summation: "<<endl;
    cin>>num1;
    cin>>num2;
    int  result = num1 + num2;
    cout<<" The sum of two numbers is : "<<result<<endl
    return 0;
}
```

Variables and Memory Concepts

- In the previous example, we had two variables (num1 , num2) where we stored the values taken from the user.
- The characters typed by the user are converted to an integer that is placed into a ***memory location*** to which the name num1 has been assigned by the C++ compiler.



Data Types

Data Type	Size	Values
int	4 Bytes	Real numbers (-15,23,0)
float	8 Bytes	Decimal numbers (0.3455)
char	1 Byte	ASCII characters (a,k,*,7,@)
bool	1 Byte	Boolean value (True – False)
string	Dynamic size	Array of characters (“Hello”)
double	8 Bytes	For double precision floating point 6e-4 = 6 * 10^(-4)

Operators

- You think of operators as a special type of function. An operator takes one or more arguments and produces a new value.

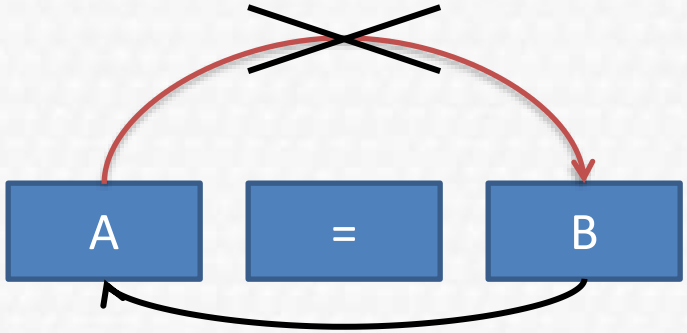
- The concepts of **addition (+)**, **subtraction (-)**, **multiplication (*), division(/)** and **assignment (=)** all have the same meaning in any programming language.

Operators

C++ Operation	C++ arithmetic operator	Algebraic expression	C++ expression
Addition	+	a + 7	a + 7
Subtraction	_	a - b	a – b
Multiplication	*	ab or a . B	a * b
Division	/	a/b , a ÷b	a/b
Modulus	%	a mod b	a % b
Assignment	=	a =b	a = b

Operators

- Some quick notes about operators:
  - The assignment in C++ is right to left.



  - If the result will be stored in the same variable of the operands it can be written like that:

    X += 1 → X = X+1

    Y *= 3 → Y = Y *3

    Z /= Y → Z = Z / Y

Operators

- The addition and subtraction can be used as a unary operator:
- X++; means (X = X + 1)
- Y--; means (Y = Y - 1)

Take Home Question:

(What is the difference between X++ and ++X ?)

Operators Precedence

Operator	Operation	Order of evaluation (precedence)
( )	Parentheses	Evaluated first. If nested, the expression in the innermost is evaluated first. If there are several evaluated from left to right.
* , / , %	Multiplication, Division, Modulus	Evaluated second. If there are several, they are evaluated left to right.
+ , -	Addition, Subtraction	Evaluated last. If there are several, the are evaluated left to right.

Simple Practice

- **Write down the order of which the operations will be evaluated:**

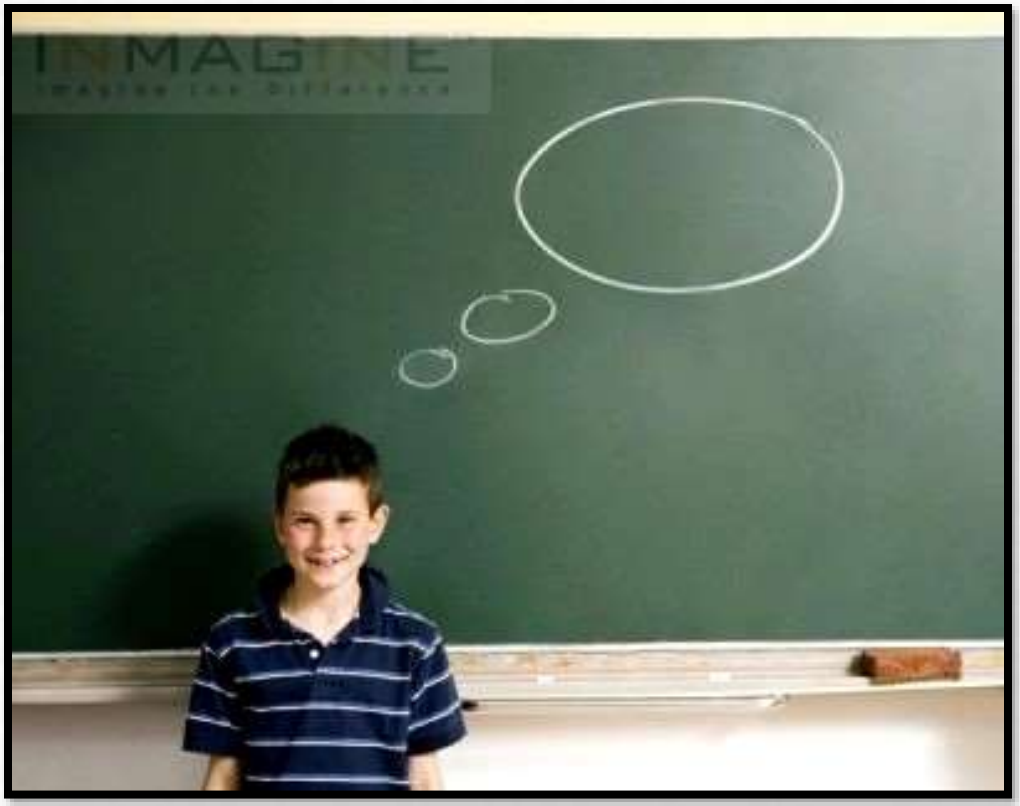
1) Z = p * r % q + w / z – y;
2) Y = m * x * x + a * x + b
3) W = (a + b * (d – e)) - m

Read and Write from Files

Take Home Question 1:

(How to use the **iostream** library to write and read from a file?)

Mind GYM



Controlling Execution

- There are many execution control statements, this includes:

- If – Else
- While
- Do – while
- For
- Switch

Conditional Statements

- **All conditional statements use the (True or False) of a condition, to determine the path of the program.**

if-else Statement

- if (expression) → True / False

  statement

//The expression decides whether to implement the statement or not

- if (expression)

  statement

  else

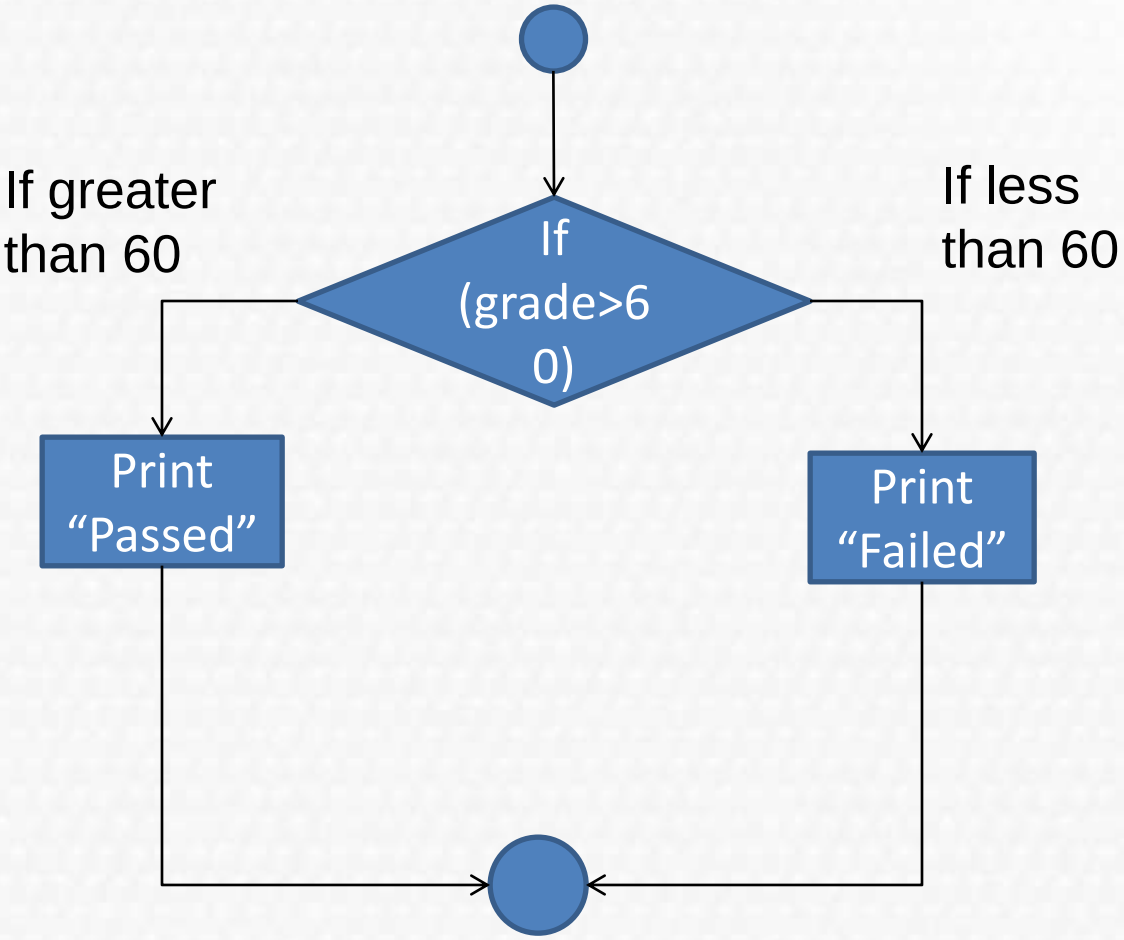
  statement

Equality and Relational Operators

C++ Operator	Sample C++ example	Meaning
>	x > y	X is greater than y
<	x < y	X is less than y
>=	x >=y	X is greater than or equal y
<=	x<=y	X is less than or equal y
==	x == y	X equal to y
!=	X != y	X not equal to y

Note: "A syntax error will occur if any of the operators ==, !=, >= and <= appears with *spaces* between its pair of symbols."

if-else Statement



if-else Statement

```
// If then.cpp : Defines the sample conditional expressions.
#include <iostream>
using namespace std;
int main()
{
   int grade;
    cout<<"Enter your grade: "<<endl;
    cin >> grade;
    if (grade>=50)
            cout<<"Congrats, You passed ;)"<<endl;
    else
            cout<<"See you next semester :( "<<endl;
}
```

Nested if-else statements

- If student's grade is greater than or equal to 90

```
    Print "A"
  Else
    If student's grade is greater than or equal to 80
      Print "B"
    Else
      If student's grade is greater than or equal to 70
        Print "C"
      Else
        If student's grade is greater than or equal to 60
            Print "D"
        Else
            Print "F"
```

Nested if-else

```
if ( studentGrade >= 90 ) // 90 and above gets "A"
   cout << "A";
else if ( studentGrade >= 80 ) // 80-89 gets "B"
   cout << "B";
else if ( studentGrade >= 70 ) // 70-79 gets "C"
   cout << "C";
else if ( studentGrade >= 60 ) // 60-69 gets "D"
   cout << "D";
else // less than 60 gets "F"
   cout << "F";
```

Dangling - Else

```
if ( x > 5 )
  if ( y > 5 )
     cout << "x and y are > 5";
else
 cout << "x is <= 5";
```

```
if ( x > 5  )
   if ( y > 5 )
      cout << "x and y are > 5";
   else
      cout << "x is <= 5";
```

These code fragments are not the same. Beware of dangling-else, so it's recommended to use braces to identify the scope of the (if-else) block.

Logical Operators

- **Logical operators** that are used to form more complex conditions by combining simple conditions. The logical operators are **&&** (logical AND), **||** (logical OR) and **!** (logical NOT, also called logical negation).

AND (&&) Operator

- Suppose that we wish to ensure that two conditions are both True before we choose a certain path of execution. In this case, we can use the **&& (logical AND)** operator, as follows:

```
if (isCar == true && speed >= 100 )
                    speedFine=400;
```

- To test whether x is greatest number from (x, y, z), you would write

```
if ( (x > y) && (y > z))
   cout<<"x is the largest number";
```

OR (II) Operator

- We use it when we have two paths, and if either one is true or both of them, a certain path of action happen.

```
if ( ( semesterAverage >= 90 ) || ( finalExam >= 90 ) )
    cout << "Student grade is A" << endl;
```

- The && operator has a higher precedence than the || operator. Both operators associate from left to right.

NOT (!) Operator

- Not Operator enables a programmer to *"reverse"* the meaning of a condition.
- The unary logical negation operator is placed before a condition when we are interested in choosing a path of execution if the original condition is false.

```
If(! (Age>=18) )
    cout<<"you can't get a driving license";

If(! (Age<=18) )
    cout<<"you can't get a driving license";
```

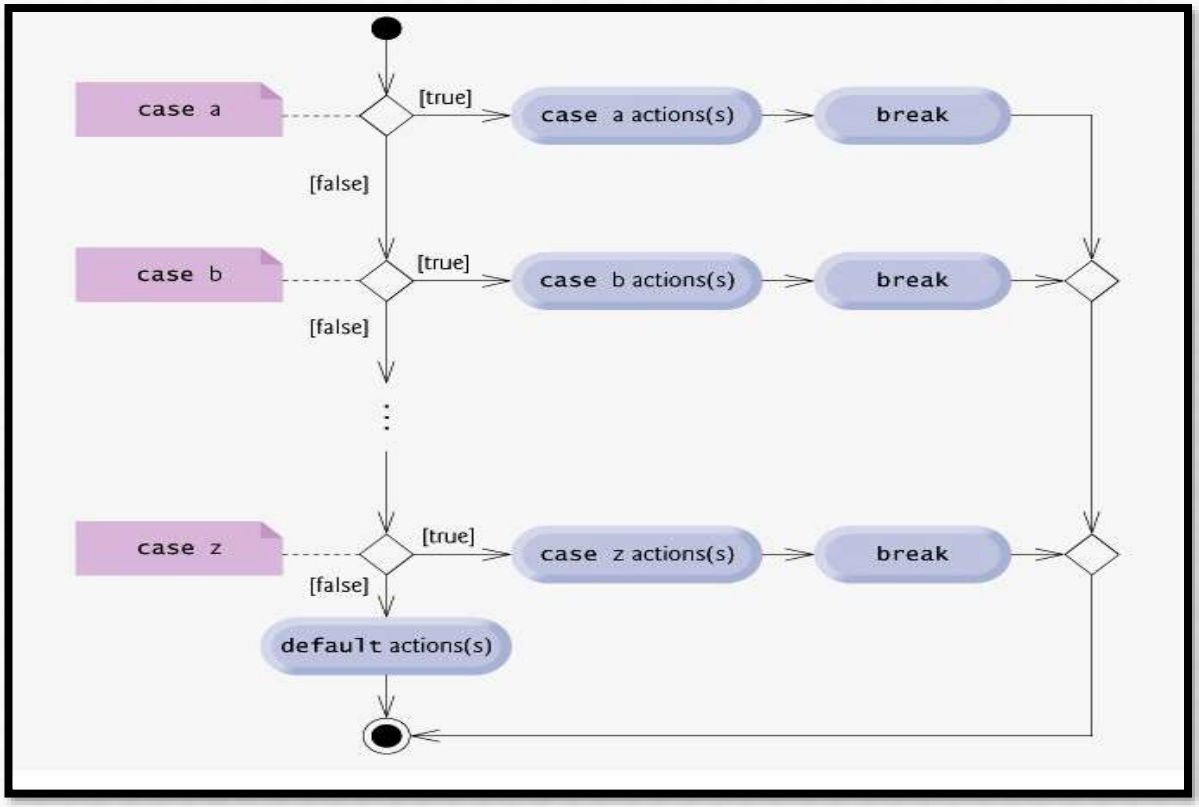
Try it out..

- What is the final result of each statement, decide whether it's (True – False):

- !(1 || 0)
- (1 || 1 && 0))
- !((1 || 0) && 0)
- !(1 == 0)

Switch Statement

- The switch provides multiple-selection statement to perform many different actions based on the possible values of a variable or expression.



Some more code..

```
switch (grade)
{
 case 'A':
            cout<<"You got A"<<endl;
            break;
 case 'B':
            cout<<"You got A"<<endl;
            break;
……….
……….
default:
   cout<<"You have failed"<<endl;
 }
```

```
Char key;
Cin>>key;

switch (key)
 {
case '+':
         add();
         break;
 case '-':
        subtract();
        break;
case '*':
        multiply();
        break;
case '/':
        divide();
        break;
default:
        Cout<<"invalid key\n";
        }
```

Some more Example..

Imagine now, you are making a software for a restaurant, and the user enters the code of the ordered food, Then prints the price of sold item.

For example:
1→ Sandwich,
2 → Juice,
3→ water ,
4→ chocolate…etc.

How you will make this using a switch statement ?

Repeat after me 1,2,3,…n ☺ (FOR.. LOOP)

- Given a number n, make a program that prints the numbers from 1 to n.

i.e. n=5

Output: 1 2 3 4 5

→What about doing it in descending order … from N down to 1 ?

So what about nested loops ?

- Let’s make a simple program that prints the multiplication table from 1 to 10 ..

Repetition Statements: (For Loop)

Print Multiplication Table

```
for(int i=1;i<=10;i++)
  {
        cout<<i<<"  ";
         for(int j=1;j<=10;j++)
         {
                cout<<i*j<<"  ";
         }
         cout<<"\n";
  }
```

Repetition Statements: (While ..)

- “Stand with anybody that stands right, stand with him while he is right and part with him when he goes wrong. “

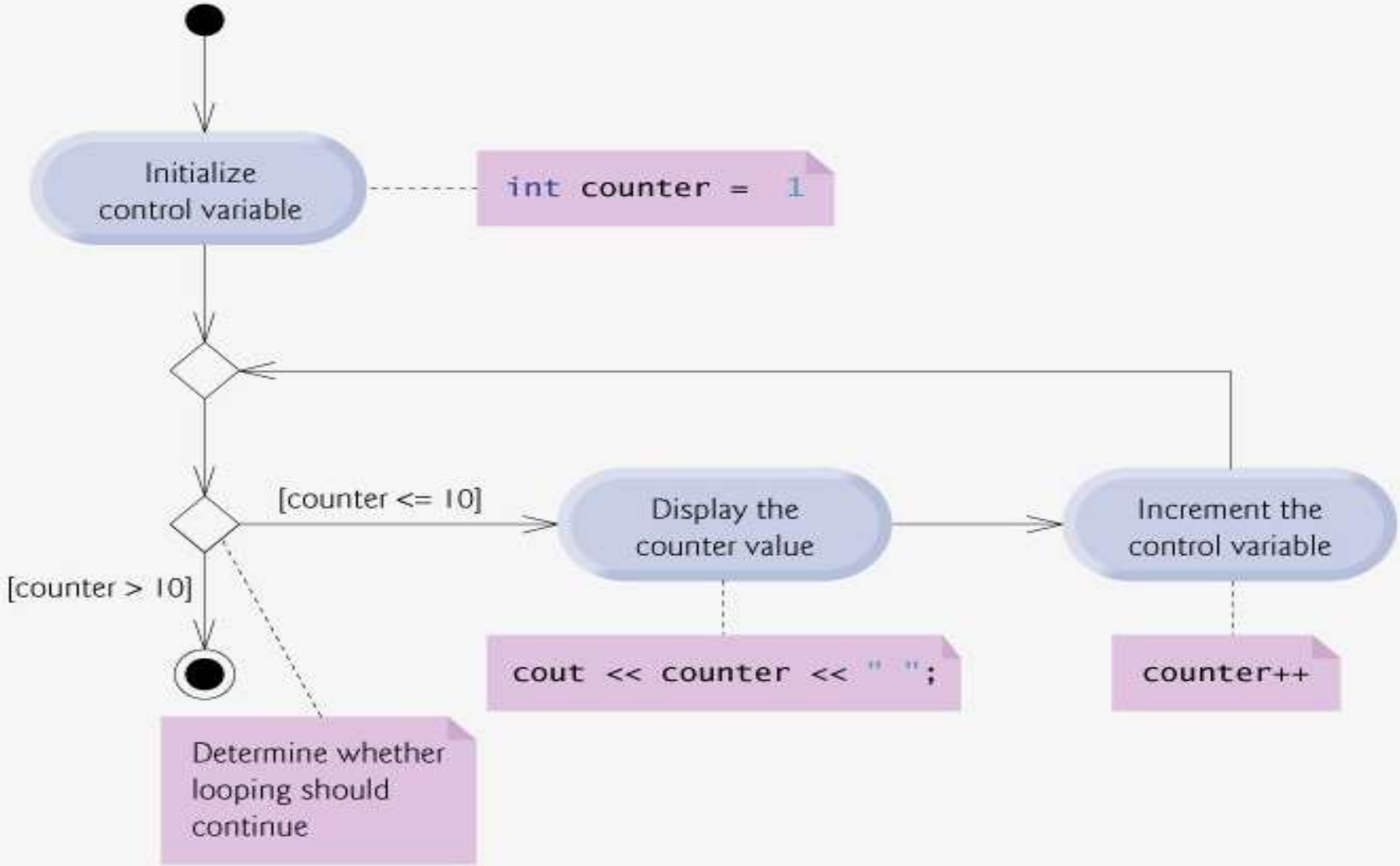
  Abraham Lincoln

- So, what’s the difference between While loop and for loop ?

While Anatomy

```
initialization;
 while ( loopContinuationCondition )
{
  statement increment;
}
```

While – Flow Chart



One of the most common usages

- We need our program to be working unless the user press n button.

```
Char c=0;
while(true)
  {
        c=getchar();
        if(c=='n')
        {
              cout<<"n key is pressed\n";
              break;
        }
   }
```

Repetition Statements: (Do-While)

- The do...while repetition statement is similar to the while statement. In the while statement, the loop-continuation condition test occurs at the beginning of the loop before the body of the loop executes. The do...while statement tests the loop-continuation condition after the loop body executes; therefore, the loop body always executes at **least once**.

Find errors in this code

Find the error(s) in each of the following:

1. For (x = 100, x >= 1, x++) cout << x << endl;

2. The following code should print whether integer value is odd or even:

```
switch ( value % 2 )
 {
case 0: cout << "Even integer" << endl;
 case 1: cout << "Odd integer" << endl;
}
```

Find errors in this code

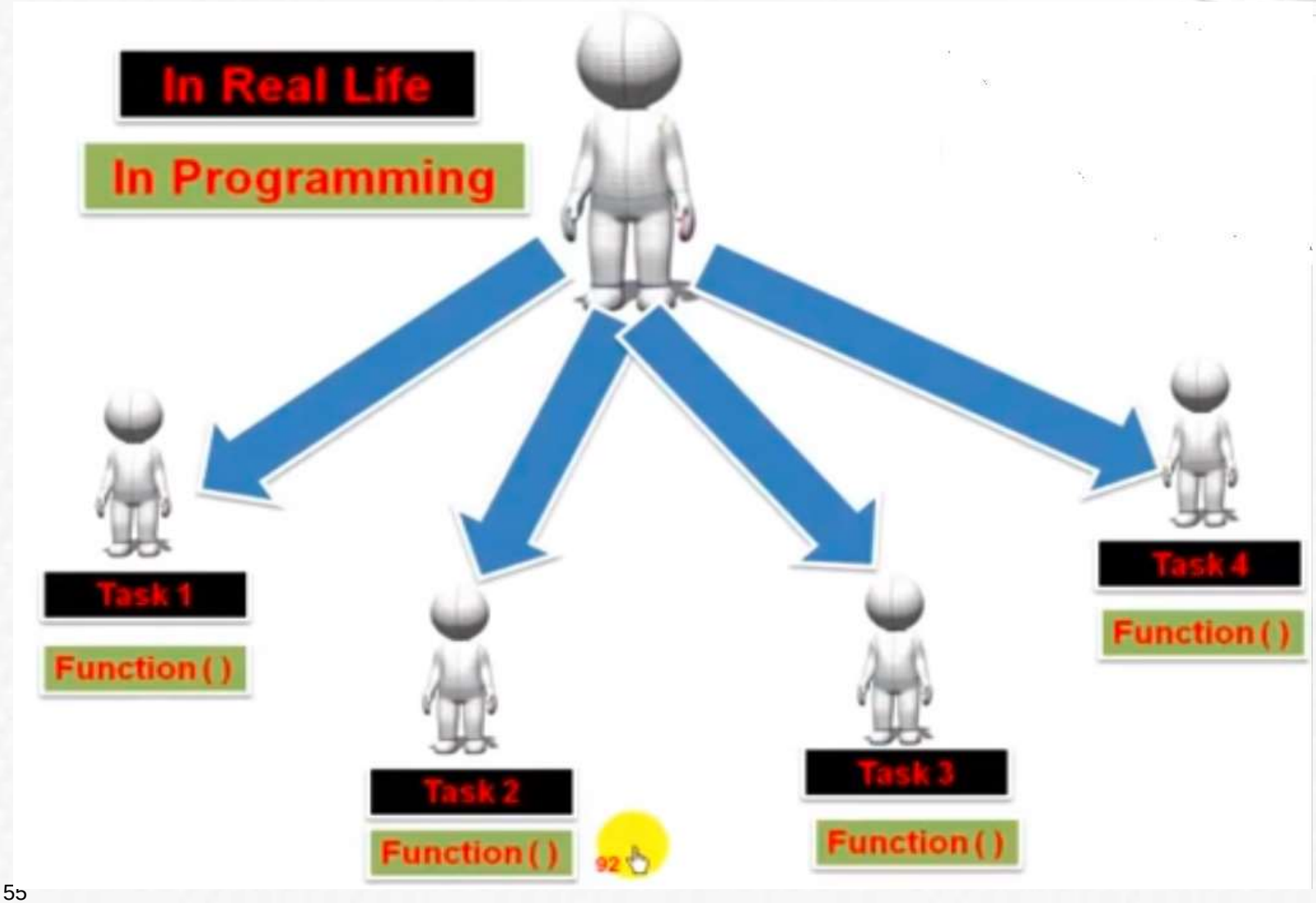
3. The following code should output the odd integers from 19 to 1:

```
for ( x = 19; x >= 1; x += 2 ) cout << x << endl;
```

4. The following code should output the even integers from 2 to 100:

```
counter = 2;
do {
cout << counter << endl;
 counter += 2;
}
 While ( counter < 100 );
```

Function


In Real Life
In Programming
Task 1
Function ( )
Task 2
Function ( )
Task 3
Function ( )
Task 4
Function ( )

Function

```
#include <iostream>
using namespace std;
Int sum (x, y)
{
    int  Z = x + y;
   return Z;
}
int main()
{
    int num1, num2, result;
    cout<<"Enter two numbers to get their summation: "<<endl;
    cin>>num1; num2;
     result = sum (num1; num2)
    cout<<" The sum of two numbers is : "<<result<<endl
    return 0;
}
```

Arrays and Pointers

Arrays

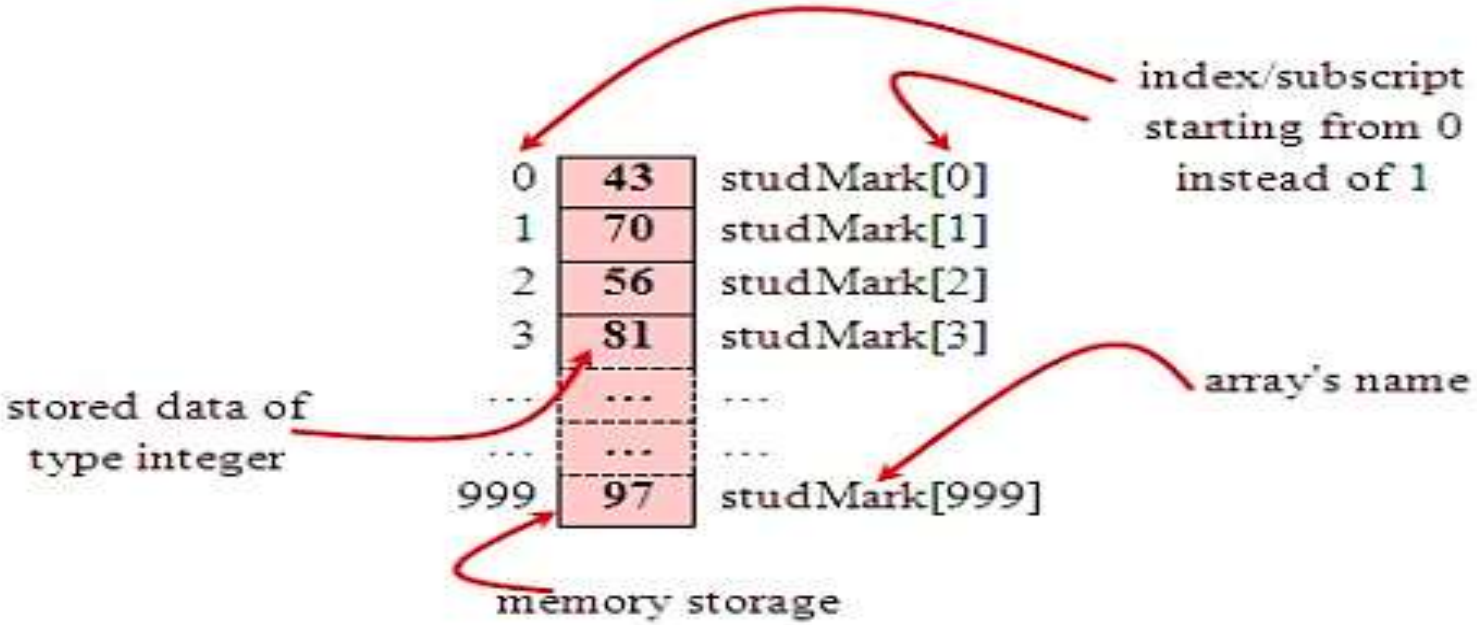
- Can you imagine how long we have to write the declaration part by using normal variable declaration?

```
int main(void)
{
   int studMark1, studMark2, studMark3,
   studMark4, …, …, studMark998, stuMark999,
   studMark1000;
   …
   …
   return 0;
}
```

Arrays

- `int studMark[1000];`

- This will reserve 1000 contiguous memory locations for storing the students' marks.
- Graphically, this can be depicted as in the following figure.



Arrays

- A collection of identically typed data items distinguished by their indices (or subscripts)
- One dimensional arrays

  type identifier[size];

  lower_bound = 0, upper bound = size –1

int ar[100];

ar[0]	ar[1]	. . .	ar[99]

Arrays

- Initializing Arrays

  type name [*elements*];

  int x[5];

  int x[3]={1,2,3};

  int x[10]={1};

must be constant

Arrays

float a[3] = { 0.1, 0.2, 0.0 };

int a[3] = { 3, 4 };

int a[] = { 2, -4, 1 };≡ int a[3] = { 2, -4, 1 };

char s[] = “abcd”; ≡ char s[] = { ‘a’,‘b’,‘c’,‘d’,‘\0’ }

Arrays

- Accessing Arrays

  array[index]

  cout << x[2];

  x[1]=100;

Arrays

- Valid operations with arrays:

```
a=5;
x[0] = a;
x[a] = 75;
b = x[a+2];
x[x[a]] = x[2] + 5;
```

Arrays

- Practice

```
 #include <iostream>
using namespace std;
int billy [] = {16, 2, 77, 40, 12071};
int n, result=0;

int main ()
{
  for ( n=0 ; n<5 ; n++ )
  {
    result += billy[n];
  }
  cout << result;
  return 0;
}
```

Pointers

Point to here,
point to there,
point to that,
point to this,
and point to nothing!
well, they are just memory addresses!!

How a computer stores information in memory

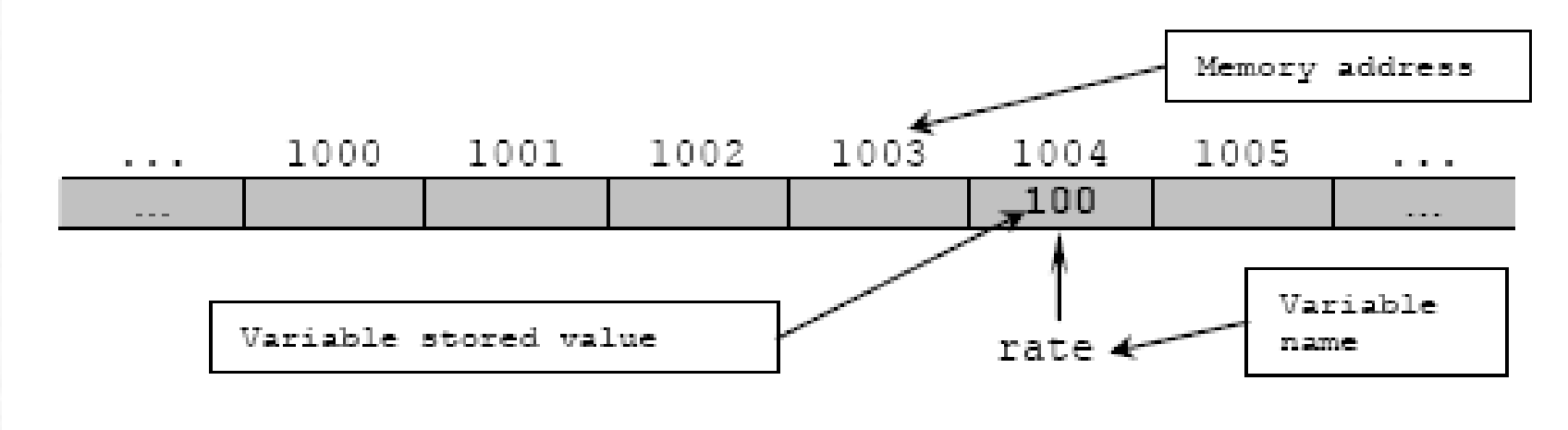
- Random Access Memory (RAM) consists thousands of sequential storage locations

- Locations are identified by a unique address.

- Addresses range from 0 to a maximum value that depends on the amount of physical memory installed.

How a computer stores information in memory

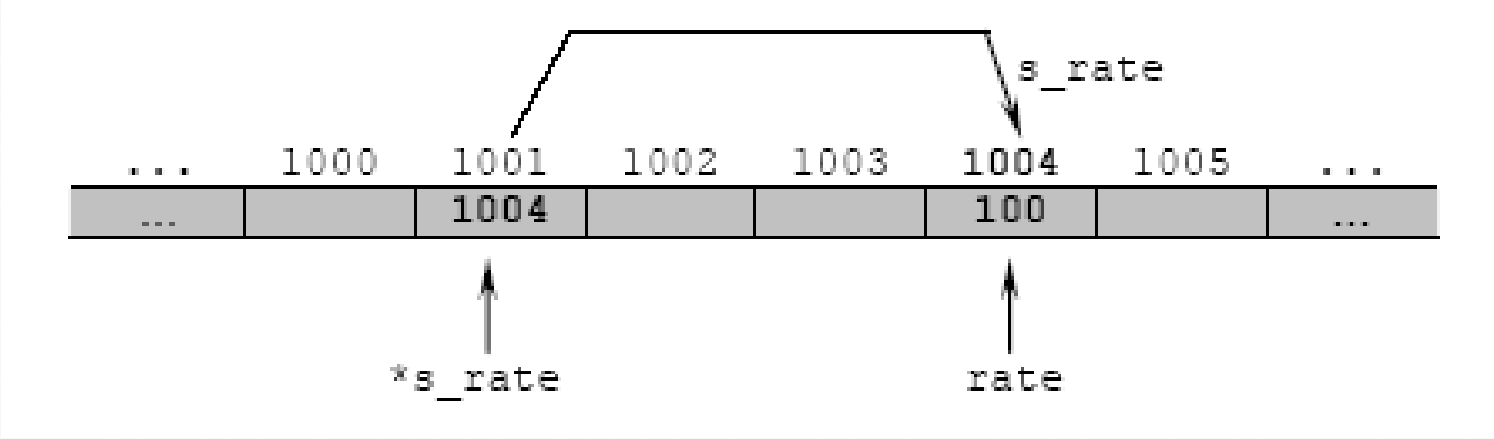
```
int   rate = 100;
```



Pointers

int *s_rate;

s_rate is pointing to rate or is said a pointer to rate



Pointers

s_rate is pointing to rate or is said a pointer to rate

Where:

address	variable name	hold data

Pointers

- A pointer is a variable that contains the memory address of another variable, where, the actual data is stored.

Importance of Pointers

•Provide a powerful and flexible method for manipulating data in programs

• Very useful for dynamic and efficient memory management programming

•Allow different sections of code to share information easily

•BUT, Pointers also generate many bugs in programs if used improperly or maliciously

Next….

- ***Static & Dynamic Allocation***
  - ✓**Data structure concept**
  - **Abstract Data Types (ADT)**
    - **List of data structure**
    - **Array**
    - **Linkedlist**
    - **Queue**
    - **Stack**
    - **Trees**